



UNIVERSIDAD POLITÉCNICA DE VICTORIA

REPORTE DE ACTIVIDADES DEL PROYECTO

**SISTEMA WEB DE GESTIÓN Y ACTUALIZACIÓN DE DATOS
DEL PERSONAL DEL PJETAM**

QUE PRESENTA

VALERIA MAYRÍN BERMÚDEZ RAGA

**EN CUMPLIMIENTO DE LA ESTADÍA TSU EN
DESARROLLO DE SOFTWARE MULTIPLATAFORMA**

**ASESOR INSTITUCIONAL
DR. SAID POLANCO MARTAGÓN**

**UNIDAD ECONÓMICA RECEPTORA:
SUPREMO TRIBUNAL DE JUSTICIA DEL ESTADO DE TAMAULIPAS**

**ASESOR DE LA UNIDAD ECONÓMICA
ING. LUIS ROBERTO FLORES DE LA FUENTE**

Ciudad Victoria, Tamaulipas, Agosto de 2026.



EDUCACIÓN
SECRETARÍA DE EDUCACIÓN PÚBLICA



Tamaulipas
Gobierno del Estado



Secretaría
de Educación



Formato de autorización para exposición de Estadía de Técnico Superior Universitario

Fecha y lugar en que se emite: **Ciudad Victoria, Tamaulipas, a 31 de Julio de 2026**

Nombre completo del estudiante: **Valeria Mayrín Bermúdez Raga**

Matrícula: **2430215**

Programa Académico: **Ingeniería en Tecnologías de la Información e Innovación Digital**

Por medio del presente, se le informa que tras realizar su Estadía en la empresa **Supremo Tribunal de Justicia del Estado de Tamaulipas** llevando a cabo el proyecto: **Sistema Web de Gestión y Actualización de Datos del Personal del PJETAM**, durante el periodo comprendido **Mayo-Agosto 2026** logrando una ponderación de ____ (60 % posibles) para el criterio de reporte; por lo tanto, se otorga la oportunidad de exponer el trabajo realizado por medio de un informe ejecutivo programado para el día **12** del mes de **Agosto** del año en curso en un horario de **10:00 hrs** en modalidad **Presencial** en la **Oficina I-XXX69**.

Sin otro particular, se le recomienda estar disponible para iniciar su exposición 10 minutos antes de la indicación oficial.

Atentamente

Dr. Said Polanco Martagón
Asesor institucional

UNIVERSIDAD POLITÉCNICA DE VICTORIA

Av. Nuevas Tecnologías 5902
Parque Científico y Tecnológico de Tamaulipas
Carretera Victoria Soto La Marina Km. 5.5
Cd. Victoria, Tamaulipas. C.P. 87138

Tel: (834) 1711100 al 10
www.upvictoria.edu.mx





Control sumativo de Estadía para Técnico Superior Universitario

Criterio	Valor / Ponderación	Calificación
Reporte	60 %	
Exposición	40 %	
Calificación final		

Datos de la evaluación sumativa

Fecha: **12 / Agosto / 2026**

Dr. Said Polanco Martagón
Asesor institucional

Resumen

■

Abstract

Contenido temático

Resumen	III
Abstract	IV
1 Introducción	1
1.1 Antecedentes	1
1.1.1 Antecedentes de la empresa	1
1.1.1.1 Información general del PJETAM	1
1.1.1.2 Misión	2
1.1.1.3 Visión	2
1.1.1.4 Tribunal Electrónico para el manejo de datos del personal	2
1.1.2 Antecedentes teóricos	3
1.1.2.1 Sistemas Heredados (Legacy Systems) en Administración Pública	3
1.1.2.2 Limitaciones Técnicas de ASP.NET Web Forms	3
1.1.2.3 Impacto de la Arquitectura Modelo-Vista-Controlador (MVC)	4
1.1.2.4 Tabla Comparativa de Tecnologías	4
1.2 Definición del Problema	5
1.3 Objetivos	6
1.3.1 Objetivo General	6
1.3.2 Objetivos Específicos	6
1.4 Justificación	7
1.5 Alcances y Limitaciones	8
1.5.1 Alcances	8
1.5.2 Limitaciones	8
2 Marco Teórico	9
2.1 Modernización de Sistemas	9
2.1.1 Sistemas Heredados (Legacy Systems)	9
2.2 Evolución de las Tecnologías de Desarrollo Web en Microsoft	10
2.2.1 La Era de ASP.NET Web Forms	11
2.2.2 Nueva Generación de la Plataforma .NET (.NET Core y .NET moderno)	12
2.3 Patrones de Diseño y Arquitectura de Software	12
2.3.1 Patrones Arquitectónicos	13
2.3.1.1 Patrón Modelo-Vista-Controlador (MVC)	13

2.4	Seguridad, Autenticación e Integración de Datos	15
2.4.1	Control de Acceso Basado en Roles (RBAC)	15
2.4.2	Integración de Sistemas mediante APIs	16
2.4.3	Persistencia y Mapeo de Datos	16
3	Metodología	18
3.1	Propuesta de Solución	18
3.1.1	Beneficios	18
3.1.2	Recursos Técnicos y de Software	19
4	Resultados	20
5	Conclusiones	21
	Bibliografía	22
	Anexos	24

1. Introducción

Actualmente las instituciones gubernamentales suelen utilizar sistemas que pueden resultar ser algo anticuados ya que la mayoría fueron desarrollados hace años y continúan usando tecnologías de esa época; a simple vista, estos pueden parecer funcionales pero varios de estos pueden presentar problemas como problemas de mantenimiento, algunas limitaciones técnicas como incompatibilidad con sistemas operativos modernos y riesgos en la seguridad de los datos.

Es necesario modernizar estos sistemas antiguos ya que mientras más pase el tiempo más cambian las necesidades y más cambian las tecnologías modernas, lo que dificulta la adopción de nuevas funciones que la empresa necesite para escalar o mejorar la atención al personal.

Tal es el caso del sistema para la gestión y actualización de datos del personal del Poder Judicial del Estado de Tamaulipas, el cual fue desarrollado hace más de 15 años, y su interfaz y funcionamiento lo demuestran. La seguridad y eficiencia en el manejo de los datos del personal garantiza la capacidad para poder mantener sus servicios funcionando y también la protección de los datos sensibles lo que ayuda a que también haya un clima de confianza entre los trabajadores y el manejo de sus datos, además de disminuir las equivocaciones asociadas a la manipulación de estos datos.

En este proyecto se busca implementar un sistema siguiendo las bases del que está ya en funcionamiento pero integrando un tipo de acceso centralizado y que pueda tener un mantenimiento sencillo a largo plazo con ayuda de desarrollo web moderno e integración de APIs, para así lograr una mejora en los procesos administrativos que realizan en el Poder Judicial del Estado de Tamaulipas.

1.1. Antecedentes

1.1.1. Antecedentes de la empresa

1.1.1.1. Información general del PJETAM

El Poder Judicial del Estado de Tamaulipas es una institución pública encargada de impartir y administrar justicia en el estado de Tamaulipas. Su objetivo principal es resolver conflictos entre particulares y autoridades, garantizando el Estado de derecho, la protección de los derechos humanos y la paz social.^[1]

Entre las principales funciones institucionales que este realiza se encuentran resolver conflictos

entre los ciudadanos, entre empresas, o entre los particulares y el Estado, basándose en la legislación vigente. De igual forma, asegurar que ninguna ley, norma o acto de autoridad vulnere lo establecido en la Constitución y servir de salvaguarda ante violaciones a los derechos humanos y libertades individuales. Además de actuar como un contrapeso democrático para limitar el poder del Ejecutivo y del Legislativo, asegurando que sus actos se mantengan dentro del marco.[1]

1.1.1.2. Misión

Impartir y administrar justicia de manera eficiente y expedita, solucionando conflictos a través de los órganos jurisdiccionales y los medios alternos, contribuyendo a garantizar el estado de derecho y la paz social en Tamaulipas.[2]

1.1.1.3. Visión

Transcender como Institución de excelencia, mediante la profesionalización e institucionalización de sus integrantes, en un ambiente de humanismo y justicia institucional, haciendo uso de la innovación tecnológica y optimización de los recursos, con responsabilidad social y pleno respeto a la ley, los derechos humanos y la igualdad de género.[2]

1.1.1.4. Tribunal Electrónico para el manejo de datos del personal

El Poder Judicial del Estado de Tamaulipas mantiene una necesidad constante de modernizar y ampliar sus sistemas informáticos para agilizar trámites y garantizar la transparencia; esto se refiere a que es primordial tener infraestructura, software especializado y herramientas de ciberseguridad de alta calidad para el desarrollo de sus sistema informáticos. Para hacer frente a estas necesidades, la institución se apoya en el Tribunal Electrónico, un ecosistema digital que requiere soporte y desarrollo tecnológico continuo.[1]

Dentro del ecosistema del Tribunal Electrónico se encuentra el sistema para la gestión y actualización de datos del personal, el cual fue desarrollado aproximadamente hace 15 años utilizando tecnologías como el framework ASP.NET Web Forms y el lenguaje de programación Visual Basic, bajo una arquitectura en capas, separando el diseño visual de la lógica del sistema. Aunque dicho sistema permitió durante varios años llevar a cabo procesos administrativos relacionados con la información del personal, con el paso del tiempo comenzaron a presentarse diversas limitaciones derivadas de la antigüedad de las tecnologías empleadas.

Entre estas limitaciones destacan las dificultades de mantenimiento, la complejidad para implementar nuevas funcionalidades, problemas de compatibilidad con herramientas modernas y restricciones en aspectos relacionados con la seguridad y escalabilidad. Esta situación evidenció

la necesidad de modernizar el sistema mediante una solución web basada en tecnologías actuales que permitieran mejorar la eficiencia operativa, la experiencia de usuario y la integración con otros servicios institucionales.

1.1.2. Antecedentes teóricos

1.1.2.1. Sistemas Heredados (Legacy Systems) en Administración Pública

El avance continuo de las tecnologías de la información provoca que sistemas informáticos que han sido utilizados por años se conviertan en sistemas heredados o legacy systems, los cuales, a pesar de presentar limitaciones técnicas u operativas, siguen vigentes porque estos tienen un alto valor y tienen datos que ya son vitales para la organización. En las organizaciones públicas, la eliminación o sustitución inmediata de estas plataformas es una tarea sumamente compleja debido al riesgo latente de interrumpir las actividades administrativas diarias y afectar directamente la continuidad de la entrega de servicios institucionales.

Con el paso de los años, mantener estos sistemas antiguos eleva considerablemente los costos de soporte técnico y aumenta la dificultad de mantenimiento debido a la escasez de expertos en herramientas obsoletas junto con una documentación de referencia que suele ser insuficiente. De acuerdo con Mandviwalla et al. (2026)[3], los sistemas heredados centrales personalizados terminan atrapando las prácticas críticas de la organización y su información en un entorno técnico moribundo y rígido. Esto no solo incrementa la vulnerabilidad ante ciberataques o caídas operativas, sino que convierte al software en una "trampa de valor" que frena la innovación interna, bloquea la agilidad institucional y dificulta la consolidación limpia de los datos.

1.1.2.2. Limitaciones Técnicas de ASP.NET Web Forms

Un ejemplo de esta problemática de obsolescencia se presenta en las aplicaciones web desarrolladas bajo el entorno de ASP.NET Web Forms. Según Herceg (2024)[4], esta tecnología de generaciones previas basaba su arquitectura en componentes de servidor y en un estado persistente oculto denominado View State, el cual provocaba transferencias de datos pesadas y complejas (postbacks) hacia el servidor ante cualquier interacción del usuario. Este flujo no solo generaba problemas de parpadeo y lentitud en las interfaces disminuyendo la usabilidad, sino que introducía un fuerte acoplamiento con los componentes del sistema operativo Windows y el servidor de aplicaciones IIS (Internet Information Services).

Microsoft determinó omitir por completo el soporte para la tecnología Web Forms en las nuevas generaciones de .NET Core y .NET moderno, enfocándose exclusivamente en el desarrollo de

plataformas más ligeras y eficientes. Herceg (2024)[4] subraya que, debido a que no existe un marco equivalente directo en las versiones actuales, la migración de un sistema basado en Web Forms implica obligatoriamente una reingeniería profunda y la reescritura total de las páginas y elementos de la interfaz de usuario utilizando nuevas tecnologías de presentación.

1.1.2.3. Impacto de la Arquitectura Modelo-Vista-Controlador (MVC)

Para evitar el código extenso, rígido y difícil de mantener en un proyecto, la ingeniería de software implementa ciertos patrones de diseño que nos ayudan al desarrollo de nuestros proyectos. Entre estas estructuras, se encuentra el patrón Modelo-Vista-Controlador (MVC) destaca por proponer una división modular de la aplicación en tres componentes: el modelo, encargado de la gestión y acceso a la base de datos; la vista, que es la presentación visual de la información al usuario; y el controlador, que procesa las solicitudes y dirige el flujo de la aplicación.

La utilización de este enfoque aporta ventajas significativas durante todo el ciclo de vida del sistema. Enríquez et al. (2023)[5] demuestran mediante su investigación que el patrón MVC tiene un impacto positivo sobre la seguridad, la interoperabilidad y la usabilidad de las soluciones tecnológicas corporativas. Al separar la lógica de presentación de la lógica de negocio, el patrón MVC permite a los desarrolladores lograr una mayor escalabilidad, facilitar la reutilización de componentes de software y reducir la complejidad en la realización de pruebas funcionales frente a otro tipo de patrones.

1.1.2.4. Tabla Comparativa de Tecnologías

Como se puede observar en la Tabla 1, se contrastan las deficiencias tecnológicas de la plataforma actual frente a las ventajas de la propuesta de rediseño, fundamentando la necesidad de realizar la modernización en el presente.

Tabla 1: Tabla Comparativa de Tecnologías y Justificación de Modernización.

Criterio de comparación	ASP.NET Web Forms (Sistema Heredado)	Arquitectura MVC en .NET Moderno (Propuesta)	Motivo de la Modernización Actual (Justificación)
Arquitectura de Software	Estructura basada en componentes de servidor rígidos y un estado persistente (View State).	Separación en tres capas independientes con responsabilidades distintas, que son: Modelo, Vista y Controlador.	Web Forms es una tecnología obsoleta que carece por completo de soporte o alternativas en las versiones actuales de .NET.
Mantenibilidad y Código	Propensión a mezclar la lógica de negocio dentro del código visual.	Alta modularidad que promueve el aislamiento de las reglas del negocio, facilitando la reutilización y evolución del código.	Reducir la alta deuda técnica acumulada por más de 15 años, disminuir los costosos gastos de soporte y agilizar la integración de nuevas funciones operativas.
Dependencia Tecnológica	Dependencia con componentes específicos de Windows y el servidor de aplicaciones IIS.	Diseñado de forma ligera para trabajar en entornos distribuidos o contenedores en la nube.	El entorno moderno exige agilidad multiplataforma y la capacidad técnica de responder velozmente a demandas globales de la institución.
Seguridad	Uso de Membership Providers desactualizados cuyos antiguos algoritmos de cifrado ya no son seguros hoy en día.	Gestión avanzada de identidades e integración segura mediante APIs modernas y controles basados en roles.	Salvaguardar de manera estricta la información sensible del personal y mitigar riesgos cibernéticos.

1.2. Definición del Problema

Actualmente, el Poder Judicial del Estado de Tamaulipas cuenta con un sistema para la gestión y actualización de datos del personal desarrollado hace más de 15 años. Debido al tiempo transcurrido desde su implementación, dicho sistema presenta diversas limitaciones tecnológicas que afectan su funcionamiento, mantenimiento y capacidad de adaptación a las necesidades actuales de la institución.

Entre las principales problemáticas identificadas se encuentran el uso de tecnologías obsoletas como el framework ASP.NET Web Forms, dificultades para realizar mantenimiento o implementar nuevas funcionalidades, limitaciones en la seguridad de la información y una experiencia de usuario poco eficiente. Asimismo, el sistema actual no cuenta con mecanismos modernos de integración con otros servicios institucionales, lo que dificulta la centralización y optimización de procesos relacionados con el personal.

Ante esta situación, surge la necesidad de desarrollar un nuevo sistema web que permita modernizar el proceso de actualización de datos del personal mediante el uso de tecnologías actuales, mecanismos de seguridad más robustos e integración con otros sistemas institucionales. De esta manera, se busca mejorar la eficiencia operativa, facilitar el mantenimiento futuro del sistema y brindar una herramienta más segura, escalable y funcional para sus usuarios.

1.3. Objetivos

1.3.1. Objetivo General

- Desarrollar un sistema web con tecnologías modernas que optimice y modernice el proceso de actualización de datos del personal del Poder Judicial del Estado de Tamaulipas, sustituyendo el sistema actual (desarrollado hace más de 15 años) por una solución más segura, eficiente y de fácil mantenimiento.

1.3.2. Objetivos Específicos

- **Objetivo 1:** Analizar y documentar los requerimientos funcionales y no funcionales del sistema actual, identificando sus limitaciones técnicas y las necesidades actuales de los usuarios.
- **Objetivo 2:** Diseñar la arquitectura del nuevo sistema web, seleccionando tecnologías modernas de desarrollo frontend y backend que garanticen escalabilidad y seguridad.
- **Objetivo 3:** Modelar y estructurar la base de datos para el almacenamiento eficiente de la información del personal, asegurando la integridad y consistencia de los datos.
- **Objetivo 4:** Desarrollar los módulos del sistema que permitan el registro, consulta, modificación y validación de los datos de los empleados mediante una interfaz web intuitiva y responsiva.

- **Objetivo 5:** Integrar el módulo de autenticación del nuevo sistema con el sistema de consulta de comprobantes de nómina del PJETAM, mediante una API, permitiendo que los empleados accedan a ambos sistemas con un único inicio de sesión.
- **Objetivo 6:** Implementar mecanismos de control de acceso basados en roles, garantizando que solo el personal autorizado pueda visualizar o modificar la información según su perfil.
- **Objetivo 7:** Realizar pruebas funcionales y de usabilidad del sistema para verificar el correcto funcionamiento de los módulos desarrollados y corregir errores antes de su puesta en marcha.
- **Objetivo 8:** Elaborar la documentación técnica y los manuales de usuario necesarios para la operación, mantenimiento y futura evolución del sistema.

1.4. Justificación

La modernización de los sistemas informáticos representa una necesidad importante para las instituciones que buscan optimizar sus procesos administrativos y garantizar la seguridad de la información que manejan. En el caso del Poder Judicial del Estado de Tamaulipas, el sistema actual de actualización de datos del personal presenta limitaciones derivadas de su antigüedad tecnológica, lo cual dificulta su mantenimiento, evolución y adaptación a las necesidades actuales.

El desarrollo de un nuevo sistema web permitirá mejorar significativamente la gestión de la información del personal mediante una plataforma más eficiente, segura y fácil de utilizar. Además, la implementación de tecnologías modernas contribuirá a facilitar futuras actualizaciones y ampliaciones del sistema, reduciendo problemas asociados con sistemas heredados o desactualizados.

Este proyecto busca aportar una solución tecnológica que beneficie tanto al personal administrativo encargado de la gestión de información como a los empleados que harán uso del sistema, promoviendo procesos más ágiles, seguros y eficientes dentro de la institución.

1.5. Alcances y Limitaciones

1.5.1. Alcances

El presente proyecto contempla el desarrollo de un sistema web para la gestión y actualización de datos del personal del Poder Judicial del Estado de Tamaulipas, el cual permitirá modernizar el sistema actual mediante el uso de tecnologías modernas de desarrollo.

El sistema incluirá funcionalidades para el registro, consulta, actualización y validación de información del personal mediante una interfaz web intuitiva y responsiva. Asimismo, contará con mecanismos de autenticación y control de acceso basados en roles, permitiendo restringir el acceso a determinadas funciones según el perfil del usuario.

1.5.2. Limitaciones

El proyecto estará enfocado exclusivamente en una plataforma web, por lo que no se desarrollará una aplicación móvil nativa. Además, el funcionamiento del sistema dependerá de la disponibilidad de conexión a internet y de la infraestructura tecnológica proporcionada por la institución.

Otra limitación corresponde al tiempo disponible, que consta de 4 meses, para el desarrollo e implementación del proyecto, lo cual puede restringir la incorporación de funcionalidades adicionales o mejoras futuras no consideradas en el alcance inicial.

2. Marco Teórico

2.1. Modernización de Sistemas

Actualmente, la tecnología avanza tan rápido que las organizaciones e instituciones no se pueden quedar atrás. Por lo tanto, actualizar los sistemas de software se ha vuelto una de las tareas más importantes para los directores y encargados de la tecnología en las empresas. El "modernizar un sistema", no se refiere a hacer un cambio de la noche a la mañana, sino a un proceso continuo de mejora para que el software no pierda su valor. Según explican los investigadores Abu Bakar, Razali y Jambari (2020), modernizar consiste en renovar esas aplicaciones viejas utilizando tecnologías actuales o estructuras más modernas. El objetivo de esto es que el sistema pueda reaccionar rápido a los cambios que necesita la organización, funcione mejor y sea mucho más seguro.[6]

2.1.1. Sistemas Heredados (Legacy Systems)

Un concepto clave en esta área son los llamados "sistemas heredados." legacy systems. Básicamente, son sistemas de información viejos que todavía se usan porque son el motor principal de las operaciones diarias de una empresa, pero que ya se quedaron atrasados respecto a las tecnologías y estándares actuales. De acuerdo con Abu Bakar, Razali y Jambari (2020), estos sistemas utilizan lenguajes de programación y herramientas de generaciones pasadas.[6] Aunque suelen dar problemas técnicos o fallas constantes, las organizaciones no los pueden dejar de usar tan fácilmente porque tienen un valor muy alto. Esto pasa porque el sistema viejo guarda las reglas del negocio, los datos históricos de muchos años y la costumbre de los usuarios que ya se la saben de memoria; apagar el sistema de golpe afectaría por completo el trabajo de todos los días. Esto se muestra en la Figura 1, donde se pueden ver los componentes que forman parte de un sistema heredado típico, incluyendo el software y reglas de negocio.

El verdadero problema es que mantener un sistema viejo con el tiempo se vuelve una pesadilla muy cara y lenta. Cada vez es más difícil encontrar programadores que conozcan herramientas antiguas, y por lo general, la documentación o los manuales para guiarse ya no existen o están incompletos. Al final, el sistema se vuelve una barrera que no deja que la institución innove o agregue nuevas funciones que los usuarios necesitan.

Sobre esto, Mandviwalla, Tiwana y Bradshaw (2026) comentan que los sistemas viejos y personalizados terminan atrapando los datos en diferentes bases de datos que no se hablan entre sí.

Legacy System Components

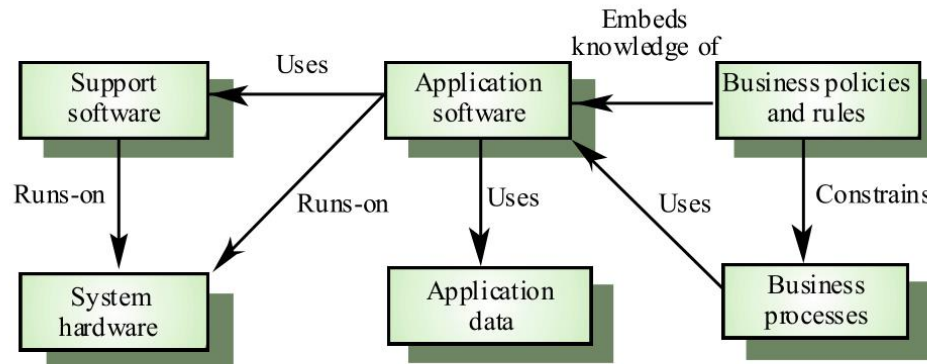


Figura 1: Componentes de un sistema heredado.

Intentar arreglar esto con soluciones rápidas o provisionales solo aumenta la "deuda técnica" (el desorden en el código), creando una trampa que eleva el riesgo de que el sistema se caiga o sufra ataques cibernéticos.[3] Por lo tanto, la mejor opción es aplicar técnicas de ingeniería inversa (revisar el código viejo para entender qué hace) y migrar la plataforma hacia un entorno moderno, recuperando la flexibilidad del software pero cuidando la información de la institución.

2.2. Evolución de las Tecnologías de Desarrollo Web en Microsoft

El desarrollo de aplicaciones web con Microsoft ha pasado por una gran transformación a lo largo de las últimas décadas. Lo que comenzó como un modelo pensado para servidores locales y sistemas operativos Windows, ha evolucionado hacia un entorno completamente abierto y diseñado para la nube. Esta evolución no solo representa un cambio de herramientas, sino un cambio completo en la manera de programar de los desarrolladores y en las metodologías usadas por las organizaciones para actualizar sus aplicaciones.

2.2.1. La Era de ASP.NET Web Forms

Lanzado a principios de la década de los 2000, ASP.NET Web Forms fue fundamental durante muchos años para la creación de sistemas empresariales y gubernamentales a gran escala. Según Zhu (2022), ASP.NET permite acelerar el desarrollo web mediante controles y funcionalidades integradas que posibilitan mostrar bases de datos y realizar tareas complejas con pocas líneas de código. Asimismo, Visual Studio incorpora herramientas visuales que facilitan la creación de interfaces mediante el arrastre y colocación de componentes.[7] En la Figura 2 se puede observar un ejemplo de una aplicación desarrollada con ASP.NET Web Forms, mostrando la interfaz de usuario y algunos de los controles disponibles para interactuar con la información.

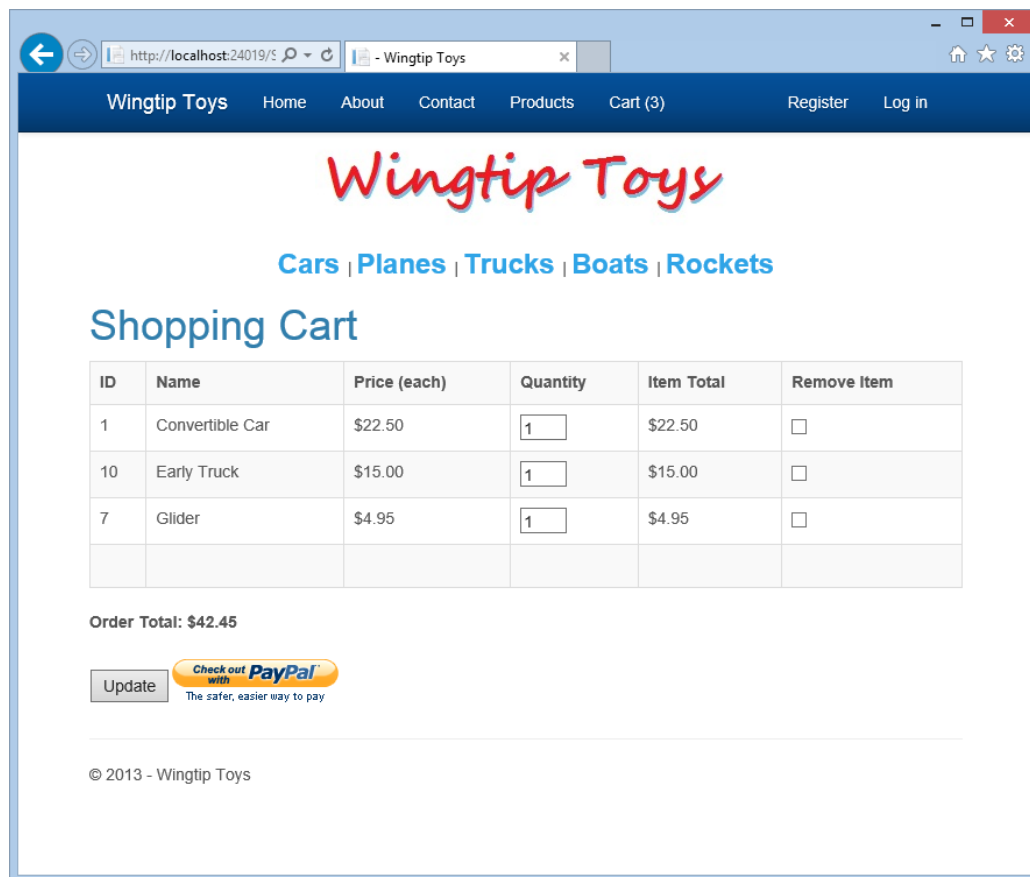


Figura 2: Ejemplo de aplicación realizada con ASP.NET Web Forms.

Sin embargo, para ofrecer una experiencia con estado sobre el protocolo HTTP, ASP.NET Web Forms incorporó el View State, un campo oculto que permitía restaurar el estado de los controles entre solicitudes al servidor. Herceg (2024) señala que este enfoque facilitaba el desarrollo de aplicaciones web, aunque también podía ocasionar tiempos de respuesta más largos, parpadeos en la interfaz y problemas de rendimiento debido al tamaño del estado almacenado en cada

petición. Además, el autor destaca la importancia de mantener separada la lógica de negocio de la presentación para simplificar el mantenimiento y futuras migraciones tecnológicas.[4]

Además, ASP.NET Web Forms mantenía una estrecha integración con Internet Information Services (IIS) y con componentes específicos del ecosistema Windows, lo que dificultaba su ejecución en otros entornos. Herceg (2024) señala que Microsoft decidió no ofrecer soporte para esta tecnología en .NET Core debido a la complejidad de reimplementarla sin introducir cambios incompatibles significativos, por lo que la modernización de estas aplicaciones requiere la reescritura de las páginas y componentes de interfaz utilizando otras tecnologías de presentación.[4]

2.2.2. Nueva Generación de la Plataforma .NET (.NET Core y .NET moderno)

La verdadera revolución llegó en el año 2016 con el lanzamiento de .NET Core 1.0, marcando el inicio de una nueva generación tecnológica. A diferencia del Framework tradicional, esta versión se diseñó desde cero para ser de código abierto, modular y completamente multiplataforma, permitiendo que el software corra tanto en Windows como en Linux o macOS. Como describe Herceg (2024), .NET 5 sucedió a .NET Core 3.1 y dio origen a la actual línea de versiones de .NET (5, 6, 7 y posteriores). Asimismo, Microsoft ha establecido que .NET Framework únicamente recibirá actualizaciones de seguridad, mientras que el desarrollo de nuevas funcionalidades y tecnologías se concentra en la rama moderna de .NET.[4]

Esta nueva plataforma destaca principalmente por su alto rendimiento y velocidad. Los ingenieros Montes Baez y Portilla Flores (2024) señalan que la adopción de arquitecturas distribuidas y microservicios en ASP.NET Core permite desarrollar, probar y desplegar componentes de manera independiente, lo que facilita la actualización de funcionalidades específicas sin necesidad de publicar todo el sistema y mejora su agilidad y escalabilidad.[8] De acuerdo con Montgomery (2025), él sostiene que la migración hacia .NET moderno favorece la adopción de arquitecturas modulares y orientadas a la nube, incrementando la flexibilidad, la capacidad de crecimiento y la adaptación de los sistemas empresariales a las necesidades tecnológicas actuales.[9]

2.3. Patrones de Diseño y Arquitectura de Software

En el ámbito de la ingeniería de software, los programadores buscan constantemente estrategias para estructurar el código de manera eficiente, optimizar la velocidad de su trabajo y mejorar la calidad de las aplicaciones. A lo largo de un proyecto, es habitual que se repitan características

similares dentro del código para elaborar elementos comunes de la interfaz. Si estas características no se gestionan correctamente, suelen duplicarse, lo que genera un código realmente extenso, desorganizado y difícil de mantener.

Para solucionar este inconveniente, se introdujeron los patrones de diseño. Aplicado en desarrollo de sistemas, los investigadores Gavilánez Alvarez, Layedra y Ramos (2022) aclaran que un patrón de diseño no consiste en un conjunto de herramientas que se instalan directamente en el código de una aplicación, sino que es un concepto y un razonamiento lógico bien definido para resolver problemas comunes de organización e interfaces.[10] Su implementación evita la duplicación de código, facilita su reutilización y asegura la validez del software al basarse en estructuras que ya han sido probadas por otros desarrolladores.

2.3.1. Patrones Arquitectónicos

Debido al gran número de patrones que existen en la actualidad, es necesario agruparlos en categorías según sus características o propósitos. En el nivel más alto de esta clasificación se encuentran los patrones arquitectónicos. Como señalan Enríquez et al. (2023), los patrones arquitectónicos definen la estructura básica de un sistema informático y funcionan como una plantilla base sobre la cual se construye una aplicación, permitiendo organizar sus componentes y responsabilidades de manera más eficiente.[5]

2.3.1.1. Patrón Modelo-Vista-Controlador (MVC)

El Modelo-Vista-Controlador, conocido popularmente como MVC, es uno de los patrones arquitectónicos más utilizados y populares en la industria del desarrollo de software. Fue descrito por primera vez por Trygve Reenskaug en el año 1979. Según la revisión bibliográfica realizada por Gavilánez Alvarez, Layedra y Ramos (2022), el propósito fundamental del MVC es clasificar la información, la lógica interna del sistema y la interfaz que se le presenta al usuario, dividiendo la aplicación en tres módulos o componentes.[10]

Los componentes son:[5]

- **Modelo:** Es el componente responsable de manipular, gestionar y actualizar directamente los datos del sistema.
- **Vista:** Se encarga de la interfaz de usuario. Su función es presentar visualmente la información en pantallas y recibir las interacciones finales del usuario.

- **Controlador:** Su función es comunicar al modelo con la vista, solicita los datos necesarios al modelo, los procesa y se los entrega a la vista para que los muestre.

En la Figura 3 se puede observar un esquema que representa la interacción entre los tres componentes del patrón MVC, mostrando cómo el controlador actúa como intermediario entre el modelo y la vista, facilitando la separación de responsabilidades y promoviendo un diseño más limpio y mantenible.

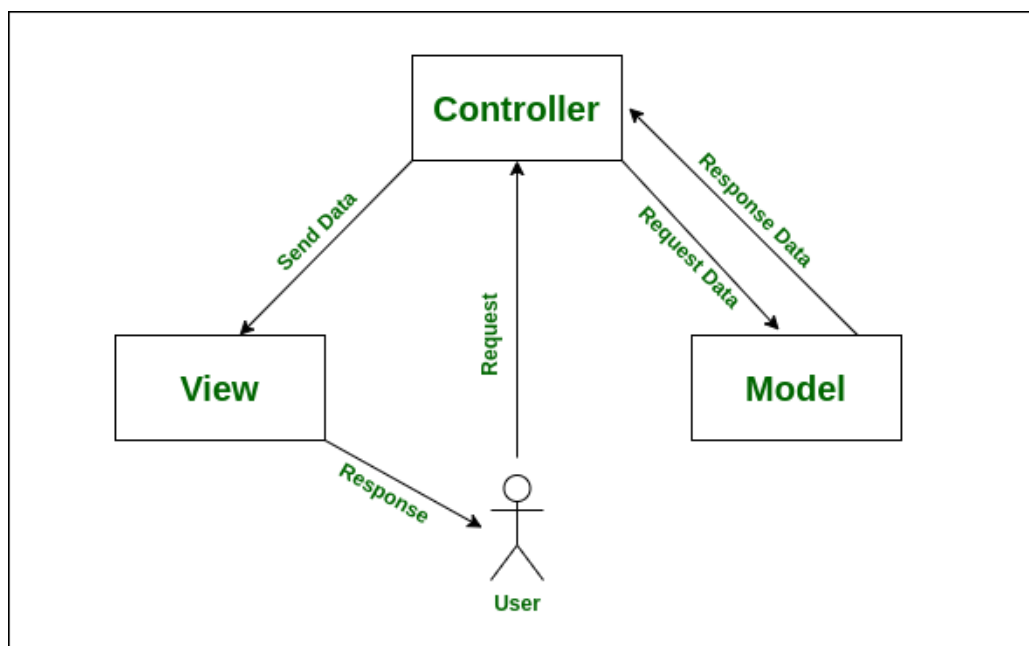


Figura 3: Representación del patrón MVC.

Enríquez et al. (2023) destacan en sus conclusiones que el patrón MVC ofrece ventajas sustanciales, ya que permite separar limpiamente la lógica de la presentación de la lógica de negocio, lo que otorga una alta robustez al software, facilita enormemente la reutilización de código y agiliza la realización de pruebas unitarias o de testing.[5] Por su parte, Giraldo Mejía, Vargas Agudelo y Garzón Gil (2021) señalan que el patrón MVC constituye una alternativa adecuada para proyectos de desarrollo web que requieren mantenibilidad, rendimiento y flexibilidad. Asimismo, destacan que definir una arquitectura antes de iniciar la codificación permite a los equipos de desarrollo reducir reprocesos y mantener una estructura organizada y mantenible a lo largo del proyecto.[11]

2.4. Seguridad, Autenticación e Integración de Datos

La transición hacia entornos modernos basados en internet y computación en la nube ha transformado la manera en que las organizaciones gestionan sus servicios. No obstante, este panorama también incluye importantes desafíos de seguridad y privacidad, ya que cualquier tipo de acceso no autorizado podría poner en riesgo los datos confidenciales de una empresa. Por ello, las organizaciones deben implementar políticas y cortafuegos eficaces que supervisen y salvaguarden de forma integral todo el ecosistema digital, garantizando la confidencialidad, control de acceso y protección de la información sensible.

2.4.1. Control de Acceso Basado en Roles (RBAC)

Para evitar violaciones de seguridad y garantizar que solo las personas autorizadas interactúen con la información, es fundamental implementar políticas de control de acceso bien estructuradas. En este ámbito, los mecanismos de autenticación y autorización desempeñan un papel crucial. De acuerdo con Lezcano Gil, Olivarez Geronimo y Mendoza de los Santos (2023), el control de acceso y la autenticación multifactorial constituyen medidas fundamentales para la protección de la información en la nube, mientras que modelos como el control de acceso basado en roles (RBAC) representan alternativas ampliamente utilizadas para restringir el acceso a los recursos únicamente a los usuarios autorizados.[12]

En la Figura 4 se representa mediante un esquema cómo el control de acceso basado en roles organiza a los usuarios en diferentes categorías según sus responsabilidades y funciones dentro de la organización, asignando permisos específicos a cada rol.

El control de acceso basado en roles permite asignar permisos específicos a los usuarios de acuerdo con su función dentro de la institución. En este sentido, Villanueva Guzmán et al. (2025) muestran que es posible establecer restricciones para que únicamente los administradores puedan modificar información crítica, como los productos del sistema, mientras que los usuarios estándar cuentan con permisos limitados de consulta, garantizando así la protección de los recursos sensibles.[13] Asimismo, Herceg (2024) señala que los antiguos proveedores de Membership y Roles de ASP.NET deben migrarse hacia ASP.NET Core Identity durante los procesos de modernización, ya que esta biblioteca emplea un formato distinto para el almacenamiento de contraseñas cifradas y proporciona mecanismos seguros para realizar la transición de las cuentas de usuario existentes.[4]

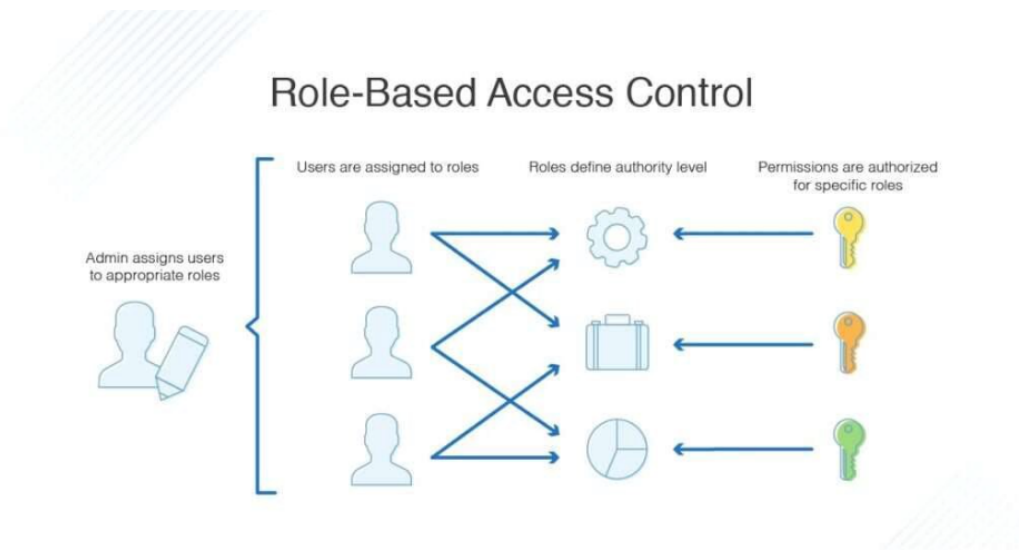


Figura 4: Representación del control de acceso basado en roles (RBAC).

2.4.2. Integración de Sistemas mediante APIs

En el contexto del desarrollo actual, la interconexión y la interoperabilidad, que es la capacidad de diferentes sistemas para compartir y utilizar información de forma coherente, pueden apoyarse en el uso de interfaces de programación de aplicaciones (APIs). Villanueva Guzmán et al. (2025) señalan que, en el desarrollo móvil y web moderno, las APIs RESTful se han convertido en una herramienta esencial para el desarrollo backend, ya que permiten conectar de forma segura y eficiente múltiples canales digitales, como aplicaciones móviles y plataformas web, facilitando el intercambio de recursos e información en tiempo real sin necesidad de diseñar continuamente nuevas infraestructuras de conectividad.[13]

A nivel arquitectónico, las APIs basadas en el estilo REST proponen una comunicación cliente-servidor sin estado que utiliza protocolos e interfaces estandarizadas para intercambiar representaciones de recursos mediante HTTP y formatos ligeros como JSON. Herceg (2024) explica que la modernización de aplicaciones web en .NET ha implicado la adopción de nuevas versiones de los frameworks tradicionales, como ASP.NET Core y ASP.NET Core Identity, los cuales fueron rediseñados para responder a las necesidades actuales de aplicaciones distribuidas y orientadas a la nube, favoreciendo soluciones más ligeras y desacopladas de infraestructuras específicas.[4]

2.4.3. Persistencia y Mapeo de Datos

El almacenamiento, procesamiento y la persistencia de la información en bases de datos relacionales constituyen el núcleo de las soluciones empresariales. Históricamente, las aplicaciones

antiguas dependían de primitivas clásicas de acceso a datos (como ADO.NET), lo que obligaba a los programadores a escribir consultas manuales extensas y lidiar con la gestión directa de las conexiones. Con la evolución técnica, cobraron una enorme relevancia los mapeadores objeto-relacional o herramientas ORM (Object-Relational Mapping), las cuales actúan como un traductor entre el modelo de objetos del código y el esquema de tablas de la base de datos relacional, tal como se representa en la Figura 5

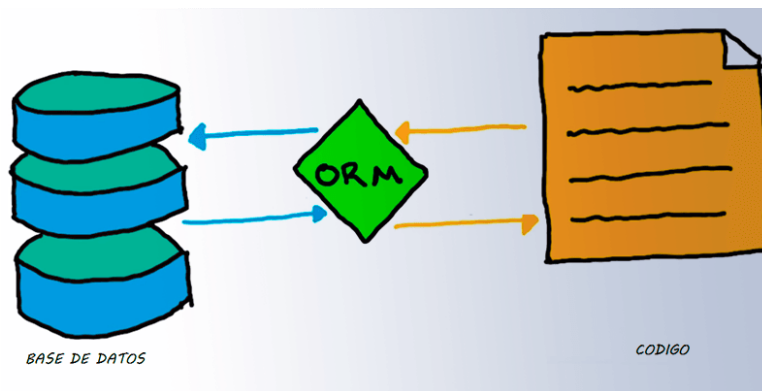


Figura 5: Representación de la relación entre código y base de datos.

En el ecosistema Microsoft, Entity Framework constituye uno de los ORM más utilizados para el acceso a datos y fue completamente rediseñado en las versiones modernas de la plataforma bajo el nombre de Entity Framework Core. Herceg (2024) explica que esta nueva implementación prioriza el rendimiento y una API más expresiva para definir y configurar el modelo de datos mediante código, incorporando enfoques inspirados en el diseño orientado al dominio y adaptándose mejor a arquitecturas modernas basadas en microservicios y entornos en la nube.[4]

3. Metodología

3.1. Propuesta de Solución

La solución propuesta consiste en el diseño, desarrollo e implementación del Sistema Web de Gestión y Actualización de Datos del Personal del PJETAM, una plataforma web moderna construida sobre el ecosistema de ASP.NET Core que sustituirá por completo al sistema heredado de más de 15 años de antigüedad. El sistema actual, desarrollado originalmente en Visual Basic y ASP.NET Web Forms (el cual se muestra en la Figura 6) ha alcanzado el final de su ciclo de vida útil, presentando una severa acumulación de deuda técnica, lentitud extrema debido al procesamiento pesado del estado de las páginas (ViewState), vulnerabilidades críticas en la seguridad de las cuentas y una rigidez estructural que impide agregar nuevas funciones indispensables.



Figura 6: Dashboard del Sistema de Actualización de Datos del Personal que se utiliza actualmente en el Poder Judicial de Tamaulipas.

3.1.1. Beneficios

- **Disminución de deuda técnica y mantenibilidad:** Se reemplaza el código monolítico y acoplado del sistema anterior por una estructura limpia basada en la separación de responsabilidades. Esto permitirá que la dirección de informática del Poder Judicial realice modificaciones, correcciones o añada nuevos módulos en el futuro de forma rápida y aislada, sin riesgo de romper otras partes del software.

- **Optimización del rendimiento:** Al eliminar los pesados mecanismos de persistencia en el cliente (ViewState) y los flujos circulares (postbacks) propios de Web Forms, la nueva aplicación web procesará las solicitudes de manera ligera y asíncrona. Esto reducirá drásticamente la carga sobre el servidor del PJETAM y el ancho de banda de la red institucional, garantizando tiempos de respuesta casi inmediatos para los empleados.
- **Fortalecimiento de la seguridad:** La propuesta incluye un esquema de seguridad que remueve los obsoletos proveedores de autenticación del sistema anterior. Se implementará un mecanismo de control de acceso basado en roles (RBAC) y cifrado avanzado de contraseñas para proteger de forma estricta los datos sensibles del personal judicial, asegurando que cada usuario visualice o modifique información exclusivamente de acuerdo con su perfil autorizado.

3.1.2. Recursos Técnicos y de Software

- **Entorno de Desarrollo (IDE):** Microsoft Visual Studio (Edición Enterprise / Community) como plataforma central de codificación.
- **Framework de Desarrollo:** .NET moderno (ASP.NET Core) configurado bajo el patrón MVC.
- **Sistema de Gestión de Base de Datos:** Microsoft SQL Server para la persistencia e integración del repositorio de datos existente.
- **Herramientas de Mapeo y Migración:** Entity Framework Core integrado mediante la consola de administración de paquetes NuGet.

4. Resultados

5. Conclusiones

Referencias

- [1] Poder Judicial del Estado de Tamaulipas. *Reglamento para el Acceso a los Servicios del Tribunal Electrónico del Poder Judicial del Estado de Tamaulipas*. https://www.tribunalelectronico.gob.mx/TE/documentos/REGLAMENTO_PARA_EL_ACCESO_A_LOS_SERVICIOS_DEL_TRIBUNAL_ELECTRONICO_DEL PODER_JUDICIAL_DEL_ESTADO_DE_TAMAULIPAS.pdf. Documento oficial consultado el 28 de mayo de 2026. 2022.
- [2] Poder Judicial del Estado de Tamaulipas. *Misión y Visión*. <https://pjetam.gob.mx/historia/mision-y-vision/>. Consultado el 28 de mayo de 2026. s.f.
- [3] Munir Mandviwalla, Amrit Tiwana y Michael Bradshaw. «Modernizing Core Legacy Systems: A Framework Based on Transplantability and Tailoring». En: *IEEE Software* (2025).
- [4] Tomáš Herceg. *Modernizing. NET Web Applications: Everything You Need to Know about Migrating ASP. NET Web Applications to the Latest Version of. NET*. Springer Nature, 2024.
- [5] Franklin Enríquez et al. «Impacto del patrón modelo vista controlador (MVC) en la seguridad, interoperabilidad y usabilidad de un sistema informático durante su ciclo de vida». En: *EASI: Engineering and Applied Sciences in Industry* 2.1 (2023), págs. 11-16.
- [6] Humairath K. M. Abu Bakar, Rozilawati Razali y Dian Indrayani Jambari. «A Guidance to Legacy Systems Modernization». En: *International Journal on Advanced Science, Engineering and Information Technology* 10.3 (jun. de 2020), págs. 1042-1050. ISSN: 2088-5334. DOI: [10.18517/ijaseit.10.3.10265](https://doi.org/10.18517/ijaseit.10.3.10265). URL: <https://doi.org/10.18517/ijaseit.10.3.10265>.
- [7] Yidong Zhu. «Diseño y desarrollo de una infraestructura web en. Net para empresa mediana o pequeña». En: (2022). URL: https://oa.upm.es/69931/1/TFG_YIDONG_ZHU.pdf.
- [8] José Cruz Montes Baez y Alberto Portilla Flores. «Migración de serverbox hacia un sistema de microservicios aplicando arquitectura vertical». En: *Revista de Investigación en Tecnologías de la Información* 12.27 Especial (2024), págs. 53-66. ISSN: 2387-0893. DOI: [10.36825/RITI.12.27.006](https://doi.org/10.36825/RITI.12.27.006). URL: <https://doi.org/10.36825/RITI.12.27.006>.
- [9] Peter A. Montgomery. «Modernizing Enterprise Web Platforms: An Evolutionary Analysis of ASP.NET to ASP.NET Core Transition Strategies». En: *The American Journal of Engineering and Technology* 7.11 (nov. de 2025). Retrieved from The American Journals, págs. 227-234. URL: <https://www.theamericanjournals.com/index.php/tajet/article/view/7383>.

- [10] Oscar Danilo Gaviláñez Alvarez, Natalia Patricia Layedra Larrea y Marco Vinicio Ramos Valencia. «Análisis comparativo de Patrones de Diseño de Software». En: *Polo del Conocimiento: Revista científico-profesional* 7.7 (2022), págs. 2146-2165.
- [11] Juan Camilo Giraldo Mejía, Fabio Alberto Vargas Agudelo y Kelly Johana Garzón Gil. «Marco de trabajo para seleccionar un patrón arquitectónico en el desarrollo de software». En: (2021).
- [12] Alfredo José Lezcano Gil, Percy Olivarez Geronimo Dionicio y Alberto Carlos Mendoza De Los Santos. «Principales medidas de seguridad para la protección de información y datos en la nube: una revisión sistemática». En: *Ingeniería Investiga* 5 (jul. de 2023), e796. ISSN: 2708-3039. DOI: [10.47796/ing.v5i0.796](https://doi.org/10.47796/ing.v5i0.796). URL: <https://doi.org/10.47796/ing.v5i0.796>.
- [13] Jorge Cein Villanueva Guzmán et al. «APIs RESTful para interoperabilidad entre aplicación Móvil y aplicación Web». En: *Ciencia Latina Revista Científica Multidisciplinar* 9.3 (jul. de 2025), págs. 6795-6821. ISSN: 2707-2215. DOI: [10.37811/cl_rcm.v9i3.18325](https://doi.org/10.37811/cl_rcm.v9i3.18325). URL: https://doi.org/10.37811/cl_rcm.v9i3.18325.

Instituto Mexicano del Seguro Social

Constancia de Vigencia de Derechos

Homoclave del trámite	Homoclave del formato	Fecha de publicación del formato en el DOF
IMSS-02-020	FF-IMSS-012	10 / 11 / 2015 DD MM AAAA

Datos Generales

NSS:	55210676874
CURP:	BERV060502MVZRGLA2
Nombre(s), primer apellido y segundo apellido:	VALERIA MAYRIN BERMUDEZ RAGA
Sexo:	Mujer
Fecha de nacimiento:	02/05/2006
Lugar de nacimiento:	VERACRUZ DE IGNACIO DE LA LLAVE

Datos de Aseguramiento

Con derecho al servicio médico:	SI
Vigente:	30/03/2026
Delegación:	VERACRUZ NORTE
UMF:	UMF 027 PAPANTLA
Turno:	MATUTINO
Consultorio:	CONSULTORIO 2
Agregado Médico:	1F2006ES

Datos de Aseguramiento

Registro Patronal	Nombre o razón social
F0543370327	UNIVERSIDAD POLITECNICA DE VICTORIA
Modalidad de Aseguramiento	Descripción de Modalidad
MODALIDAD 32	SEGURO FACULTATIVO ESTUDIANTES

Detalle de vigencia

Estado	Inicio de Vigencia	Fecha de Constancia
VIGENTE	07/03/2025	30/03/2026

Beneficiarios

NO APLICA

"De conformidad con los artículos 4 y 69-M, fracción V de la Ley Federal de Procedimiento Administrativo, los formatos para solicitar trámites y servicios deberán publicarse en el Diario Oficial de la Federación (DOF)"

**GOBIERNO DE
MÉXICO****Contacto**

Paseo de la Reforma 476, P.B.
Col. Juárez, Alcaldía Cuauhtémoc
C.P. 06600, Ciudad de México.
Tel. 800 623 23 23
<http://www.imss.gob.mx/contacto>

Ciudad Victoria,
Tamaulipas, a 01
de Mayo
de 2026

SUPREMO TRIBUNAL DE JUSTICIA DEL ESTADO DE TAMAULIPAS
LUIS ROBERTO FLORES DE LA FUENTE
PRESENTE

La Universidad Politécnica de Victoria tiene a bien presentar a **BERMUDEZ RAGA VALERIA MAYRIN** estudiante del programa académico de **INGENIERÍA EN TECNOLOGÍAS DE LA INFORMACIÓN E INNOVACIÓN DIGITAL** en su modalidad de **DESARROLLO DE SOFTWARE MULTIPLATAFORMA**, con número de matrícula **2430215** y seguro facultativo IMSS número **55210676874**; quien deberá realizar su práctica profesional de **ESTADÍA TSU**, a partir del 04 de Mayo de 2026 al 14 de Agosto de 2026, con duración de **600 horas** desarrollando el proyecto **"SISTEMA WEB DE GESTIÓN Y ACTUALIZACIÓN DE DATOS DEL PERSONAL DEL PJETAM"** que le fue asignado.

Al concluir se le extenderá la carta de liberación al evaluarlo satisfactoriamente y además le solicitaremos su valiosa opinión respondiendo el formulario que se le enviará por correo electrónico, referente al desempeño del practicante, la información que nos proporcione es de vital importancia para la mejora de los programas académicos que ofrece la Universidad Politécnica de Victoria para la formación de profesionistas altamente especializados.

El practicante deberá cumplir con el Reglamento Interno aplicable al personal en su centro de trabajo.

Sin otro particular.



ATENTAMENTE



OTHÓN CANO GARZA
DIRECTOR DE VINCULACIÓN

UNIVERSIDAD POLITÉCNICA DE VICTORIA

Av. Nuevas Tecnologías 5902
Parque Científico y Tecnológico de Tamaulipas
Carretera Victoria Soto La Marina Km. 5.5
Cd. Victoria, Tamaulipas. C.P. 87138

Tel: (834) 1711100 al 10
www.upvictoria.edu.mx





PODER
JUDICIAL DE
TAMAULIPAS



Oficio núm. DI/0409/2026

Ciudad Victoria, Tamaulipas, 11 de mayo de 2026

OTHÓN CANO GARZA

Director de Vinculación de la

Universidad Politécnica de Victoria

Presente.

Por medio del presente oficio se informa que la **C. BERMUDEZ RAGA VALERIA MAYRIN**, estudiante del programa académico **INGENIERÍA EN TECNOLOGÍAS DE LA INFORMACIÓN E INNOVACIÓN DIGITAL** de la **UNIVERSIDAD POLITÉCNICA DE VICTORIA**, es aceptada en la Dirección de Informática, con el fin de realizar su Estadía TSU a partir del 04 de mayo al 14 de agosto del presente con un horario de 08:00 a 16:00 hrs.

Sin otro particular por el momento y agradeciendo de antemano las atenciones brindadas a la presente, le envío un cordial saludo.

PODER JUDICIAL
DE TAMAULIPAS



ÓRGANO DE ADMINISTRACIÓN
JUDICIAL
DIRECCIÓN DE INFORMÁTICA
CD. VICTORIA, TAMAULIPAS

ATENTAMENTE

ING. LUIS ROBERTO FLORES DE LA FUENTE

Oficial Judicial B

Vo.Bo.

ING. ARSENIO ARMANDO CANTÚ GARZA

Director de Informática

C.C.P. ARCHIVO

ING'AACG/cr

Dirección de Informática



Boulevard Praxedís Balboa # 2207 entre López Velarde y Díaz Mirón
Col Miguel Hidalgo Ciudad Victoria, Tamaulipas. C. P. 87090



834 318 7130



arsenio.cantu@pjetam.gob.mx



Ing. Juan Camaney de Alba Rojas
PRESENTE

INSERTAR AQUÍ
DOCUMENTO
DE CARTA
DE LIBERACION
DEBIDAMENTE
FIRMADO

Rúbrica para la Evaluación del Reporte de Estadía

Nombre completo del estudiante: Valeria Mayrín Bermúdez Raga Matrícula: 2430215 PERIODO: Mayo-Agosto 2026 Nombre del evaluador: Dr. Said Polanco Martagón			
Firma del Evaluador: _____ Calificación Final: ____ / 100			
Valor	Características a cumplir	Puntos	Observaciones
6	Resumen ejecutivo y abstract: Indica qué se		
6	Contenido temático: Describe correctamente el contenido del documento		
6	Introducción: El informe cumple con introducir al lector de una manera atractiva el panorama general del trabajo realizado en la unidad económica.		
6	Descripción de la unidad económica: Se redactan los detalles de los antecedentes del departamento, la empresa, giro, misión, visión, infraestructura, ubicación.		
6	Objetivos operativos: Describen las acciones a cumplir mediante actividades establecidas por el asesor empresarial		
24	Metodología: Descripción a detalle de las etapas y procedimientos utilizados en el proyecto		
18	Resultados: Interpreta y analiza los resultados obtenidos durante su estadía.		
12	Conclusiones: Las conclusiones son claras y acordes con el objetivo esperado		
6	Bibliografía y anexos: Cita correctamente las fuentes de información utilizando un estilo de referencia; anexa cartas correspondientes		
10	Responsabilidad: Entregó el reporte en la fecha y hora señalada. Asistió al menos al 80% de sus días de Estadía		

Rúbrica para Exposición del Reporte de Estadía

Criterio	Nivel de logro		
	Bueno (10 %)	Regular (7 %)	Menor a lo esperado (5 %)
Dominio del tema (10 %)	Utiliza todos los términos y conceptos de manera correcta.	Utiliza la mayoría términos y conceptos de manera correcta	Confunde los términos y conceptos.
Presentación de material visual (10 %)	Las diapositivas usadas tienen fondo claro, letras, tablas e imágenes visibles. La presentación contiene los aspectos relevantes de la Estadía	Las diapositivas usadas presentan dificultad para su apreciación visual. La presentación contiene los aspectos relevantes de la Estadía	Las diapositivas usadas presentan dificultad para su apreciación visual. La presentación carece de algunos aspectos relevantes de la Estadía.
Comunicación y expresión (10 %)	Su volumen de voz es comprensible. Su expresión corporal manifiesta seguridad ejecutiva. Su atuendo cumple los estándares de ser formal o semiformal. Mantiene una línea de respeto en general.	Su volumen de voz no es comprensible. Su expresión corporal manifiesta inseguridad. Su atuendo cumple los estándares de ser formal o semiformal. Mantiene una línea de respeto en general.	Su volumen de voz no es comprensible. Su expresión corporal manifiesta inseguridad. Su atuendo no cumple los estándares de ser formal o semiformal. Realiza alguna falta de respeto.
Respuestas (10 %)	Las respuestas son consistentes y con parámetros de lógica operativa.	Las respuestas no son consistentes pero tienen parámetros de lógica operativa.	Las respuestas son difusas e incoherentes.
Ponderación final de su exposición:		___ /100	
Día / Mes / Año		12/ Agosto/ 2026	
Comentario adicional			